Dynamic Shooting from Static Lookup Tables, a Probabilistic Approach: TrajectorySolver

Aymen Abbassi
Gus Barry

March 2026

1 Introduction

This project details the implementation of a shoot on the move algorithm for the 2026 FIRST Robotics game: Rebuilt. In the game, a robot's objective is to shoot yellow balls known as "fuel" into the "hub" located on their respective sides of the field. To maximize throughput into the hub, our team believed that shooting while intaking balls at the same time would maximize efficiency. Our goal was to be able to shoot and, regardless of the velocity or heading of the robot, be able to score.

To shoot anywhere on the field into the hub, we construct three lookup tables for flywheel speed, angle above the horizontal, and the time lookup table that gives time-of-flight of the projectile. All are based on distance to the target, and are constructed statically when the robot is stationary. Empirical measurements are taken and the data is interpolated to produce linear and cubic functions for the time-of-flight and the speed and angle tables, respectively. Let $v, \phi, \text{tof} : \mathbb{R} \to \mathbb{R}$ be the interpolated speed, angle, and time-of-flight lookup tables respectively.

2 Localization/State Estimation

The lookup tables need distance to calculate the desired setpoints, so we require the position of the robot on the field. There are two ways in which the pose of the robot is determined.

Wheel odometry: The chassis consists of a swerve-based drive system, with 4 drive motors and 4 steer motors. Encoders on the motor provide wheel deltas over time that are integrated every cycle to give robot translation. An inertial measurement unit provides yaw/heading. This pose estimation model is the system model in the Kalman filter. In addition, it also provides the robot-relative velocity, which is then rotated by the heading θ to produce robot velocity

in the field frame $\mathbf{v}_r = \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}$.

Vision Measurements: AprilTags/ChArUco tags are used on the field that cameras use to calculate robot translation and rotation, relative to the field origin. This pose estimation model is the measurement model in the Kalman filter.

These measurements are combined in a latency-adjusted Kalman filter to produce the fused robot position in the field-frame $\mathbf{p}_r = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$ with associated covariance matrix $P \in \mathbb{R}^{3\times 3}$.

3 Algorithm

The projectile inherits the robot's velocity while shooting on the move, introducing additional displacement not accounted for in the lookup tables. To use the tables, we need to factor in this added displacement and subtract it from the position of the target.

The rotational components for both position and velocity are not needed since only translation is needed for the projected target, so the position and velocity terms reduce to $\mathbf{p}_r = \begin{bmatrix} x & y \end{bmatrix}^T$ and $\mathbf{v}_r = \begin{bmatrix} v_x & v_y \end{bmatrix}^T$ respectively.

The term $\mathbf{v}_r t_f$ represents the displacement vector of the ball in its trajectory inherited from the initial velocity of the robot, where t_f is the time-of-flight. Thus, we need to subtract this term from the current target pose so it "cancels out" the extra displacement the ball experiences in the air. Let $\mathbf{p}_{hub} = \begin{bmatrix} x & y \end{bmatrix}^T$ be the constant position of the target in the field frame: the hub. Then, the robot's position relative to the target is:

$$\mathbf{r}(t) = \mathbf{p}_r - \mathbf{p}_{hub} - \mathbf{v}_r t$$

This gives the effective target position in the target frame, and then we take the norm to obtain our distance function:

$$d(t) = ||\mathbf{p}_r - \mathbf{p}_{hub} - \mathbf{v}_r t||$$

To find the tof, we lookup:

$$t_f = \text{tof}(d)$$

However, d depends on t_f. The time-of-flight depends on distance, but distance also depends on time-of-flight, creating a recursive dependence. To solve this, define the residual:

$$E(t) = t - \text{tof}(d(t))$$

And the solution to:

$$E(t) = 0$$

Will be our time-of-flight t_f. Newton's method is a fast way to solve this equation iteratively. The recursive newton iteration is given by:

$$t_{n+1} = t_n - \frac{E(t_n)}{E'(t_n)}$$

Taking derivatives:

$$E'(t) = 1 - \text{tof}'(d(t))d'(t)$$

$$d(t) = ||\mathbf{r}(t)|| \implies d'(t) = \frac{\mathbf{r}(t) \cdot \mathbf{r}'(t)}{||\mathbf{r}(t)||}$$

Since:

$$\mathbf{r}'(t) = -\mathbf{v}_r$$

We get:

$$d'(t) = -\frac{(\mathbf{p}_r - \mathbf{p}_{hub} - \mathbf{v}_r t) \cdot \mathbf{v}_r}{d(t)}$$

$$E'(t) = 1 + \text{tof}'(d(t))\frac{(\mathbf{p}_r - \mathbf{p}_{hub} - \mathbf{v}_r t) \cdot \mathbf{v}_r}{d(t)}$$

With the final update:

$$t_{n+1} = t_n - \frac{t_n - \text{tof}(d(t_n))}{1 + \text{tof}'(d(t_n))\frac{(\mathbf{p}_r - \mathbf{p}_{hub} - \mathbf{v}_r t_n) \cdot \mathbf{v}_r}{d(t_n)}}$$

As an initial guess, we use the current time-of-flight estimate from the current position:

$$t_0 = \text{tof}(||\mathbf{p}_r - \mathbf{p}_{hub}||)$$

We repeat Newton's update step until convergence, usually taking 3-5 iterations. The result is the final time-of-flight t_f. We plug this into the distance equation and lookup our setpoints for the target speed and angle:

$$v_{target} = v(d(t_f))$$

$$\phi_{target} = \phi(d(t_f))$$

4 Propagation of Uncertainty

The fused robot pose used to calculate the projected target carries some amount of covariance P, so our resulting projected distance may also carry uncertainty. Furthermore, since distance also does not rely on rotation, the covariance matrix of $\mathbf{p}_r$ reduces to the 2×2 block matrix containing the covariance between only x and y: P_{xy}. After the Newton iterations, we treat the resulting time-of-flight t_f as locally constant and it is used to calculate projected distance:

$$d(\mathbf{p}_r) = ||\mathbf{p}_r - \mathbf{p}_{hub} - \mathbf{v}_r t_f|| = \sqrt{(\mathbf{p}_r - \mathbf{p}_{hub} - \mathbf{v}_r t_f)^T (\mathbf{p}_r - \mathbf{p}_{hub} - \mathbf{v}_r t_f)}$$

Define $\mathbf{r} = \mathbf{p}_r - \mathbf{p}_{hub} - \mathbf{v}_r t_f$ to be the projected target.

To propagate our uncertainty, we rely on a first-order Taylor expansion to linearize the distance function. In this case, variances are usually small, and the time-of-flight equation is linear and therefore smooth, so this approximation is appropriate:

$$\text{Cov}(d) = J\Sigma_{\mathbf{p}_r} J^T = J P_{xy} J^T$$

The Jacobian of our distance function is given by:

$$J_d = \frac{\mathbf{r}^T}{||\mathbf{r}||} = \frac{1}{d(\mathbf{p}_r)} \mathbf{r}^T$$

Now we propagate the variance through:

$$\sigma_d^2 = \frac{1}{d(\mathbf{p}_r)^2} \mathbf{r}^T P_{xy} \mathbf{r}$$

The linearized function for $d(\mathbf{p}_r)$ is a linear transformation of a Gaussian variable $\mathbf{p}_r$ and is therefore approximately Gaussian:

$$d \sim \mathcal{N}(||\mathbf{r}||, \sigma_d^2)$$

The propagated variance provides a measure of uncertainty in the projected shot distance. With a point estimate $\hat{d} = ||\mathbf{r}||$, we can calculate the probability of falling within a certain acceptable distance tolerance ϵ_d using the standard normal model:

$$p(\text{hit}) = P(|d - \hat{d}| < \epsilon_d)$$

And if $p(\text{hit}) < p_{\min}$ where $p_{\min}$ is the minimum acceptable probability, then the projected distance carries too much uncertainty to reliably fall within the acceptable range of a successful shot, and the shot is rejected.

5 Simulation

To test and refine the algorithm, a simulated projectile environment was created to model the trajectory of the ball through its flight path. The model accounts for the primary forces acting on the projectile.

Gravity continuously acts on the projectile and is given by:

$$\mathbf{F}_g = \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix}$$

Next, we consider air resistance that acts opposite to velocity and is given by:

$$\mathbf{F}_a = -\frac{1}{2} C_d \rho A |\mathbf{v}|^2 \hat{\mathbf{v}}$$

Where $\hat{\mathbf{v}}$ is the unit vector of velocity, scaled by $|\mathbf{v}|^2$. Since the coefficients $C_d \rho A$ are intrinsic to the ball and air density is assumed constant, let $C = \frac{1}{2} C_d \rho A$:

$$\mathbf{F}_a = -C|\mathbf{v}|^2 \hat{\mathbf{v}}$$

Spin on the ball induces the Magnus effect. This lift force can be modeled as:

$$\mathbf{F}_m = S(\omega \times \mathbf{v})$$

Where ω is the angular velocity of the ball and S is the Magnus coefficient that depends on ball properties and air conditions.

Thus we have our final equations of motion for the projectile:

$$\ddot{\mathbf{p}} = \mathbf{g} - C|\mathbf{v}|^2 \hat{\mathbf{v}} + \frac{S}{m}(\omega \times \mathbf{v})$$

The above equation cannot be solved analytically however, so we resort to numerically solving using forward-integration. The system is reformulated in state-space form by defining the state vector

$$\mathbf{x} = \begin{bmatrix} \mathbf{p} \\ \mathbf{v} \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \end{bmatrix}$$

The corresponding first-order dynamics are given by

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{p}} \\ \dot{\mathbf{v}} \end{bmatrix} = \begin{bmatrix} \mathbf{v} \\ \mathbf{g} - \frac{C}{m}\|\mathbf{v}\|\mathbf{v} + \frac{S}{m}(\omega \times \mathbf{v}) \end{bmatrix}$$

Expanding component-wise,

$$\dot{x} = v_x, \quad \dot{y} = v_y, \quad \dot{z} = v_z,$$

$$\dot{v}_x = -\frac{C}{m}\|\mathbf{v}\| v_x + \frac{S}{m}(\omega \times \mathbf{v})_x, \quad \dot{v}_y = -\frac{C}{m}\|\mathbf{v}\| v_y + \frac{S}{m}(\omega \times \mathbf{v})_y, \quad \dot{v}_z = -g - \frac{C}{m}\|\mathbf{v}\| v_z + \frac{S}{m}(\omega \times \mathbf{v})_z$$

To numerically integrate the state-space system, a fourth-order Runge–Kutta (RK4) method is employed due to its improved accuracy and stability compared to first-order schemes.

Let the system be written compactly as

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}),$$

with state vector

$$\mathbf{x} = \begin{bmatrix} \mathbf{p} \\ \mathbf{v} \end{bmatrix}$$

Given a time step Δt, the RK4 update from step n to $n+1$ is computed as

$$\mathbf{k}_1 = \mathbf{f}(\mathbf{x}_n),$$

$$\mathbf{k}_2 = \mathbf{f}\left(\mathbf{x}_n + \frac{\Delta t}{2}\mathbf{k}_1\right),$$

$$\mathbf{k}_3 = \mathbf{f}\left(\mathbf{x}_n + \frac{\Delta t}{2}\mathbf{k}_2\right),$$

$$\mathbf{k}_4 = \mathbf{f}\left(\mathbf{x}_n + \Delta t\mathbf{k}_3\right),$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{\Delta t}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right).$$

The simulation runs the Runge-Kutta iteration until the projectile hits the ground ($z \leq 0$). Each iteration, the position is recorded in a list, and the result is a list of x,y,z positions that can be visualized in a trajectory